

**Scaling the Maximum L_2 Regularisation Strength with Width in a
Fixed-Layer Neural Network**

Norran Yu

November 13th, 2025

Abstract

The focus of this research is to discover the impact of L2 regularization strength (λ) as the network's width increases – for a fixed-layer neural network. Data involved in the research is primary and collected using machine learning experiments. Results showed the relationship between maximum λ and network width to have an exponentially negative relationship. With a correlation coefficient of -0.982, the relationship is concluded to be very strong. In addition, the measurements of spread of the networks' evaluated losses and accuracies reveal that smaller networks are more stable than larger ones – inferred through the trend of increase in interquartile ranges and standard deviation with network width. Furthermore, a coefficient of determination of 0.836 is found for the relationship between network width and the positive impact of L2 at reducing its magnitude of overfit – concluding that approximately 84% of the effectiveness in overfit reduction can be attributed to an increase in network width.

Table of Contents

Section 1: Terminology	4
Section 2: Introduction	6
The Bias-Variance Tradeoff	7
Section 3: Method	8
Section 4: Data Analysis	9
4.1 - Measurements of Spread	9
4.2 - Measurements of Central Tendency for Magnitude of Overfitting	15
4.3 - Diagnosing Non-linearity With Residuals	18
Section 5: Conclusion	20
Section 6: References and Bibliography	21
Section 7: Appendices	23
7.1 - Appendix A	24
7.2 - Appendix B	24

Section 1: Terminology

This documents makes use of the following terms:

Model:

A model – in computation – is a trained program that uses patterns from a training dataset to make predictions or decisions on new data. The model is not explicitly programmed for every possible scenario.

Model width:

A model's width is the measure of its network's horizontal size, defined by the maximum number of neurons in any single layer of the network – excluding the input layer (Pehlevan et al., 2023). More width typically increases the capacity of each layer to process input features (such as those in image recognition).

Model depth:

A model's depth is the measure of its network's vertical size, defined by the total number of layers containing adjustable parameters (e.g. weight matrices and bias vectors). More depth allows the network to learn more complex, hierarchical feature representations (Inamdar, 2025).

Overfitting model:

An overfitting model learns its training data too well to the point of memorizing the smallest details, leading to poor performance on new, unseen data (Yıldırım, 2020). Its failure at generalisation is evident through high training accuracy but low evaluated accuracy. Hence, overfit models have a large generalisation gap.

Underfitting model:

An underfitting model refers to one that is too simple to pick up any meaningful patterns in the data (Kumar, 2025). It misses the key relationship between datapoints.

Bias:

The systematic error due to the model making overly simplistic assumptions about the data. High bias suggests the model is too simple (e.g. too shallow or narrow) and underfits the training data – consistently missing the true underlying relationships.

Variance:

The random error due to the model being overly sensitive to small fluctuations or details in the training data. High variance suggests the model is too complex (e.g. too deep or wide) and overfits the training data – performing well on the training set but poorly on unseen data.

Binary classification:

A specific type of prediction problem where the output must be one of two discrete possibilities (e.g. yes/no, spam/not spam, cat/dog, 0/1) (McCaffrey, 2023). The final layer of the network typically uses a Sigmoid activation function (Topper, 2023).

Neural network:

A neural network is a computational model inspired by the structure of the brain, composed of interconnected layers of nodes (neurons). It learns complex patterns by adjusting the weights and biases of its connections during training to transform an input (e.g. an image) into a desired output (e.g. the classification of the image).

Optimization algorithm/function (optimizer):

A training/optimization algorithm, or optimizer, is the mathematical procedure used during the training phase of a machine learning model to iteratively adjust its internal parameters (weights and biases) in the direction that minimizes the loss function.

Accuracy:

Accuracy is a performance metric that measures the overall correctness of the model's predictions ($Accuracy = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$).

Loss (value and function):

The loss value is returned by a mathematical function that quantifies the difference between a model's predicted output and the actual target value (Alake, 2024). The training algorithm attempts to minimize the loss value; usually by gradient descent.

Training:

Training is the process of adjusting the model's weights and biases using an optimization algorithm to minimize the loss function on the training data. The goal is to make the model's predictions as accurate as possible on this known data (Kromydas, 2023).

Weight (training):

A weight is a numerical value that determines the strength or importance of a connection between a previous layer's input and a neuron in the current layer. It is adjusted by the optimisation function during training.

Bias (training):

A bias is a numerical value that is added to the weighted sum of inputs. It is independent of the input data and determines the point at which the neuron starts to activate. It can either give the neuron more or less importance during calculations. Like weights, bases are also adjusted by the optimizer.

Evaluation:

Evaluation is the process of testing the trained model's performance on a separate, unseen dataset (test set or validation set) to measure how well it generalizes (Srivastava, 2025).

Generalisation Gap:

The difference between a model's performance on the training data (e.g. training loss/error) and its performance on unseen data (e.g. validation/test loss/error).

Optimal model:

The optimal model is the one that achieves the best possible generalization performance by finding the right balance in the bias-variance tradeoff (Zhu, 2022).

Regularisation:

Regularization is a technique used specifically to nudge a model toward becoming optimal by managing its complexity, primarily by reducing variance and overfitting (Vera, 2024).

L2 (regularisation):

L2 – also known as *Ridge Regression* – is a type of regularisation used in machine learning to prevent overfitting by adding a penalty term to the model's loss function. This limits the complexity of the model. Further explanations are located in the introduction.

Section 2: Introduction

In the recent decade, rapid advancements of deep learning and artificial intelligence have made neural networks the dominant tool across a vast range of complex tasks, from image recognition to natural language processing (Lee, 2024). One of the most notable capabilities of these networks is their capacity to model highly non-linear relationships – driving their success in classic machine learning problems such as binary classification (McCaffrey, 2023). However, the high capacity of complex neural networks also introduces the challenge of finding an optimal model configuration.



Figure 2.0.1 A group of oranges with water drops on them. Credit: Anthropic Claude 3 at 123RF.

Figure 2.1 provides an example of an input during the training of a standard image classification model. An underfitting model will pick up only the most obvious features: the color orange and circular shape. Hence, it is too general and will classify anything circular and orange-colored as an orange. On the contrary, a model that is overfitting picks up excessively minute details: orange, circle, water droplets, leaf, green dot, hemisphere, etc. This causes it to only be able to identify specific oranges that are wet and have leaves beside them. It will not generalise well on new, real-world data.

2.1 - The Bias-Variance Tradeoff

The ultimate goal in machine learning is to find the optimal balance in the *bias-variance tradeoff*, where the model does not overfit or underfit (Zhu, 2022). The following table organises bias and variance in relation to a model's performance:

	Overfitting	Underfitting	Type of Error
Bias	Low (model is complex enough to capture trends)	High (model is too simple and misses the relevant relations/features)	Systematic (consistent mistakes across all predictions)
Variance	High (model has learned noise/detail too closely from the training data)	Low (model is rigid and doesn't change much between different datasets)	Random (inconsistency in prediction based on specific training instances)

Table 2.1.1 Bias-variance tradeoff in relation to overfitting and underfitting.

To find the optimal balance between bias and variance, regularization is used (Vera, 2024). Specifically, this report will look at the effects of L2 regularisation on a networks' performance and the point at which it irreversibly underfits because of an overly large λ . In a complex model, some weights and biases might end up having extremely large coefficients, meaning the model relies too heavily on those specific weights, leading to overfitting. L2 regularization penalizes these large coefficients, encouraging the model to use smaller, more evenly distributed weights across all features. The following equation shows how a regularised/new loss is calculated with L2 (Kumar, 2022):

$$\text{Regularised Loss} = \text{Original Loss} + \lambda \sum_{j=1}^m w_j^2$$

- w_j are the weights (coefficients) of the model's features
- λ is the regularisation parameter. A larger λ means a stronger penalty, forcing the weights closer to zero

- $\sum_{j=1}^m w_j^2$ is the L2 norm (squared to ensure positivity) – the sum of the weights squared

Current literature in this field often conflates implicit regularization (i.e. the model’s architecture and optimizer) with explicit regularization (e.g. L2), resulting from experiments where multiple independent regularization variables are simultaneously active and changing (Zhao, 2023). This practice yields results where the contribution of L2 to generalization is ambiguous, making it difficult to establish a precise, causal relationship. This study addresses this by isolating λ via separating the effect of L2 regularization from other factors; ensuring network width is the only varying architectural element. Furthermore, current advice for finding the desired λ value is often trial-and-error (Jang, 2022). Thus, finding a systematic, data-driven rule that determines the maximum λ value before underfitting – without ambiguity and through statistics – would be a practical contribution to the field of neural networks.

Section 3: Method

To conduct this study, a neural network with 4 layers – 1 input, 2 hidden, and 1 output – is constructed (Yu, 2025); as shown below. The network’s depth is a control variable while the number of

```
x = GlobalAveragePooling1D(name="pooling_layer")(x)

x = Dense(NEURONS, activation="relu", kernel_regularizer=l2(regularizer_strength), name="dense_1_l2")(x)
x = tf.keras.layers.Dropout(DROPOUT)(x)
x = Dense(NEURONS, activation="relu", kernel_regularizer=l2(regularizer_strength), name="dense_2_l2")(x)
x = tf.keras.layers.Dropout(DROPOUT)(x)
x = Dense(NEURONS, activation="relu", kernel_regularizer=l2(regularizer_strength), name="dense_3_l2")(x)

outputs = Dense(1, activation="sigmoid", name="output_layer")(x)

model = Model(inputs=inputs, outputs=outputs, name=f"L2_Classifier_R_{regularizer_strength}")
return model
```

Figure 3.1 The fixed, 4-layered network used during training

neurons in the input and hidden layer (network width) is the independent variable. A total of five different widths are used: 64, 128, 256, 512, and 1024. Powers of 2 are commonly used in machine learning to optimize for the hardware components used during training – such as the GPU which is “built around binary addressing” (Maity, 2025). The dataset – another control variable – is the *IMDB Movie Reviews* with a size of 50 000 datapoints; a classical benchmarking dataset for binary classification. It is evenly split between negative and positive reviews,

allowing it to provide a clear indication of underfitting when accuracy reaches 50%. Furthermore, *TensorFlow* (Abadi et al., 2015) is the machine learning framework used during testing.

For each of these five networks, the L2 value is the only regulariser. Starting from zero, it increases with an increment of 0.0001 per training iteration. The “maximum L2 value” – point of no return where further regularization causes consistent underfitting – is determined by three or more consecutive instances of underfitting (accuracy being 50% for the binary classification task). This value is then recorded and used in the second round of testing. By making sure there are at least 3 consecutive instances of underfitting after the obtained value, potential outliers are eliminated. This is based on the conventions of the single-subject experimental design (SSED) where “a minimum of three baseline data points are required to establish dependent measure stability” (Kazdin, 2010).

A second test is then conducted for further precision. The test starts from the maximum λ value and increments by $\frac{\text{incrementation of first test}}{10}$ until *maximum λ value* + 1.

When finished, the program saves the data into a .json file and is then loaded into Google Sheets via *Google Sheets API*¹. This ensures speed and minimises human errors when copying over the data.

Section 4: Data Analysis

4.1 - Measurements of Spread

Figures 4.1.1 and 4.1.2 below show data after the first/general test. Since the task is binary classification – determination of whether the review is positive or negative – and the dataset is split evenly, an accuracy of around 50% indicates severe underfitting where the model is unable to identify patterns and turns to random guessing. The “maximum L2/ λ value” is the last, functional regularisation strength before three consecutive instances² of severe underfitting – three instances show irreversibility, where the model is on the “knife’s edge” of the bias-variance tradeoff.

¹ Google. (n.d.). Google Sheets API. Retrieved from <https://developers.google.com/sheets/api/>

² See **section 3** for reasons on why three consecutive instances is chosen

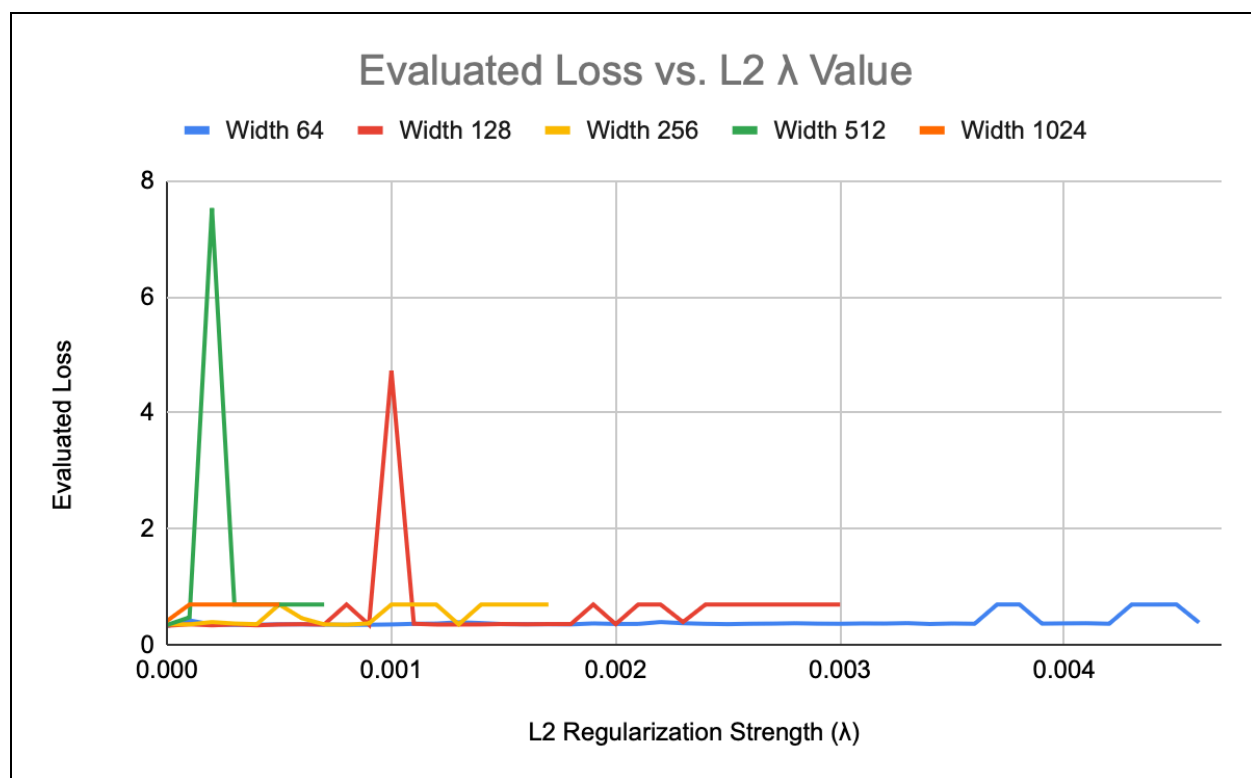


Figure 4.1.1A General test for all network widths - evaluated loss value versus λ

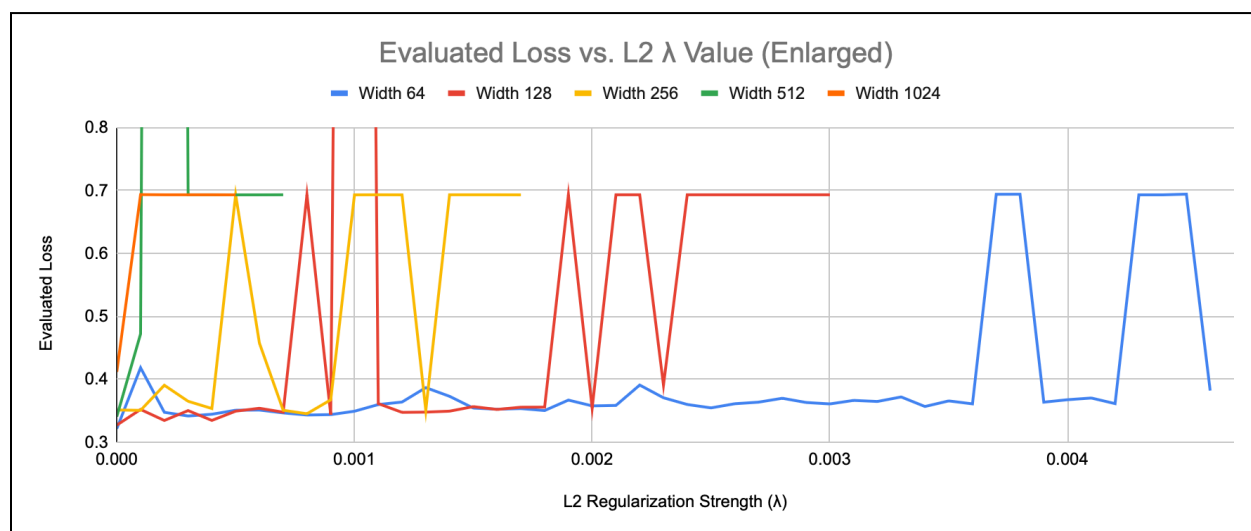


Figure 4.1.1B Enlarged version of figure 4.1.1A to show details

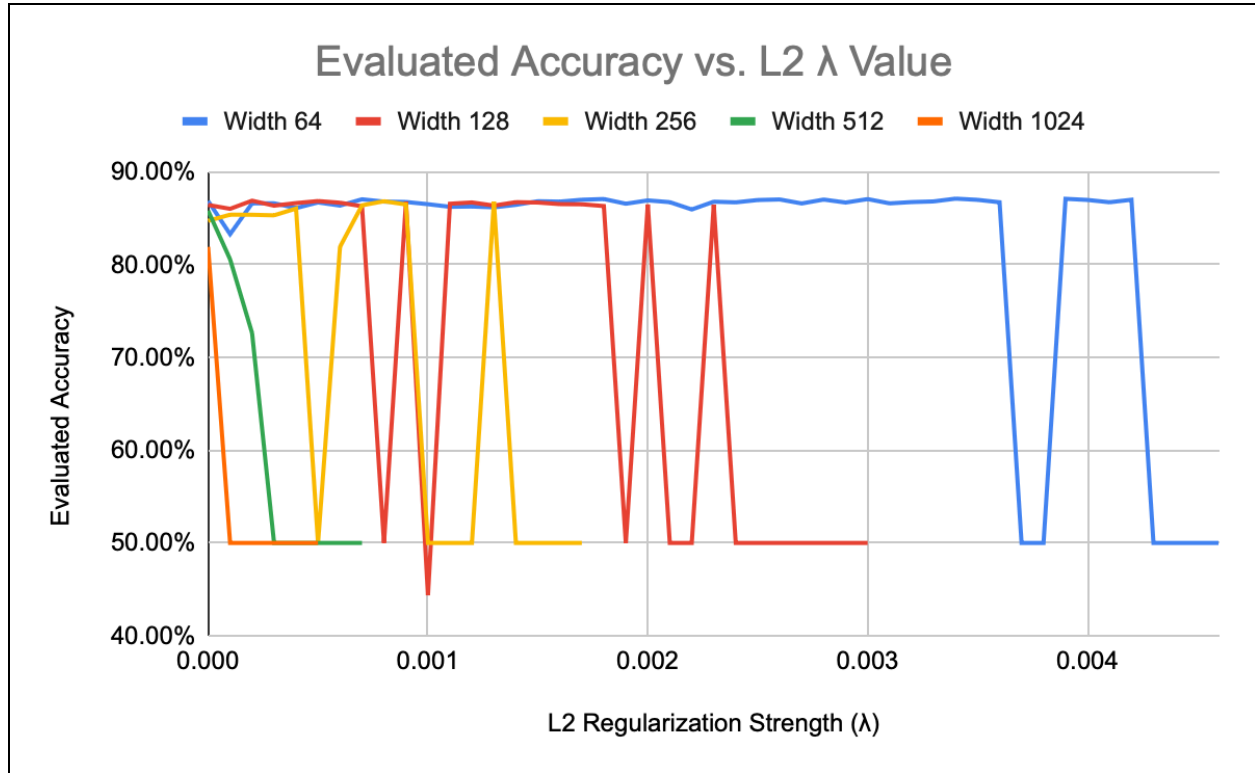


Figure 4.1.2 General test for all network widths - evaluated accuracy versus λ

As shown by the graphs above, loss value and accuracy are inversely correlated – meaning that as the loss increases, the model's accuracy generally decreases, and vice versa. An explanation to this is that if the model's predictions are consistently far-off, a large percentage of those predictions will be incorrect, resulting in a low accuracy.

The following tables only measure spread before the severely underfitting data points³ to focus on the “safe zone”. Additionally, the amount of trailing instances of irreversible underfitting is arbitrary and differs from network to network. This is due to the networks' training being stopped at different times after the underfit exhibits irreversibility.

Neurons per Layer	Q1	Median	Q3	IQR	Lower Fence	Upper Fence	Range	Outliers (λ , loss)
64	0.3518	0.3609	0.3672	0.0154	0.3287	0.3903	0.3726	(0.0000, 0.3215) (0.0037, 0.6940) (0.0038, 0.6941) (0.0041, 0.4183)
128	0.3478	0.3530	0.3686	0.0208	0.3166	0.3998	4.3993	(0.0008, 0.6931) (0.001, 4.7264)

³ All networks use data from the general test except for width 1024 since its first test has too little data to show any significance. Hence, the specific test data for 1024 is used instead. This is true for all calculations in section 4.1.

								(0.0019, 0.6933) (0.0021, 0.6932) (0.0022, 0.6932)
256	0.3508	0.3593	0.3847	0.0339	0.2999	0.4356	0.3426	(0.0005, 0.6932) (0.0008, 0.4575)
512	0.3736	0.4064	0.4393	0.0657	0.2750	0.5379	7.1901	(0.0002, 7.5308)
1024	0.3643	0.3793	0.5741	0.2098	0.0496	0.8888	1.8091	(0.00003, 2.1622)

Table 4.1.1 Measurements of spread for **figures 4.1.1**

Neurons per Layer	Q1	Median	Q3	IQR	Lower Fence	Upper Fence	Range	Outliers (λ , accuracy)
64	0.8655	0.8674	0.8698	0.0043	0.8590	0.8763	0.3712	(0.0001, 0.8329) (0.0037, 0.5000) (0.0038, 0.5000)
128	0.8623	0.8648	0.8669	0.0046	0.8554	0.8738	0.4254	(0.0008, 0.5000) (0.001, 0.4436) (0.0019, 0.5000) (0.0021, 0.5000) (0.0022, 0.5000)
256	0.8488	0.8539	0.8631	0.0143	0.8273	0.8846	0.3684	(0.0005, 0.5000)
512	0.8189	0.8320	0.8451	0.0262	0.7796	0.8844	0.1319	(0.0002, 0.7263)
1024	0.6513	0.8442	0.8506	0.1993	0.3523	1.1496	0.3594	None

Table 4.1.2 Measurements of spread for **figure 4.1.2**

The values show that the interquartile range increases dramatically with the number of neurons per layer. This infers that there is a much higher variance and instability in the model's final performance; thus, resulting in a wider range of possible outcomes as the network size increases. The following figure shows this relationship using a side-by-side boxplot. Networks with widths of 64, 128, 256, 512, and 1024 go from bottom to top, respectively.

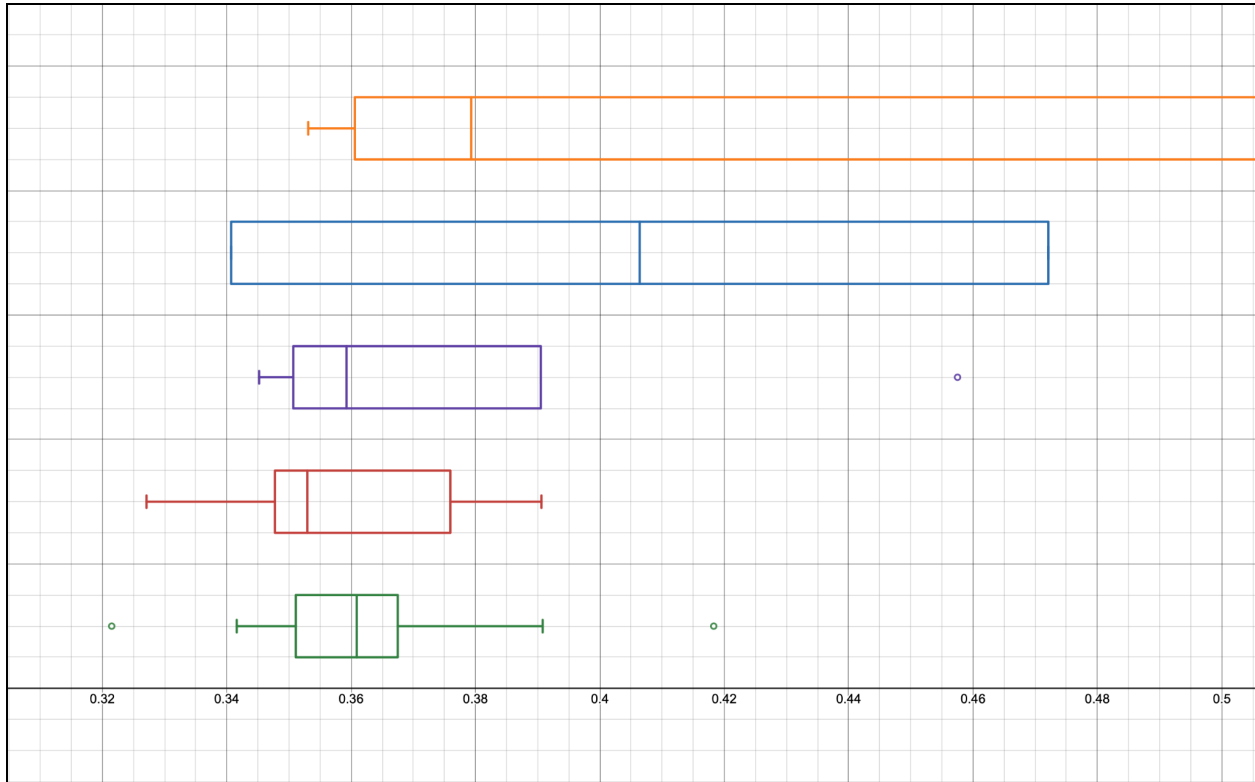


Figure 4.1.3 Boxplots of evaluated loss of networks

Many outliers in **table 4.1.2** – where accuracy reaches around 50% – are sporadic occurrences of severe underfitting. However, they are too few in a row to be marked as a robust indication of irreversibility in underfitting and failure in the bias-variance tradeoff. **Figure 4.1.4** below visualises this phenomenon, where there are sporadic drops around $\lambda = 0.0009$ and 0.0021 . The maximum λ is located at 0.0023 because of the three or more consecutive, trailing drops in accuracy afterwards. Furthermore, extremely large loss value outliers in **table 4.1.1** (bolded values) can be explained by the model overshooting when trying to minimize its loss during gradient descent – going past the minimum and ending up at another loss high-point.

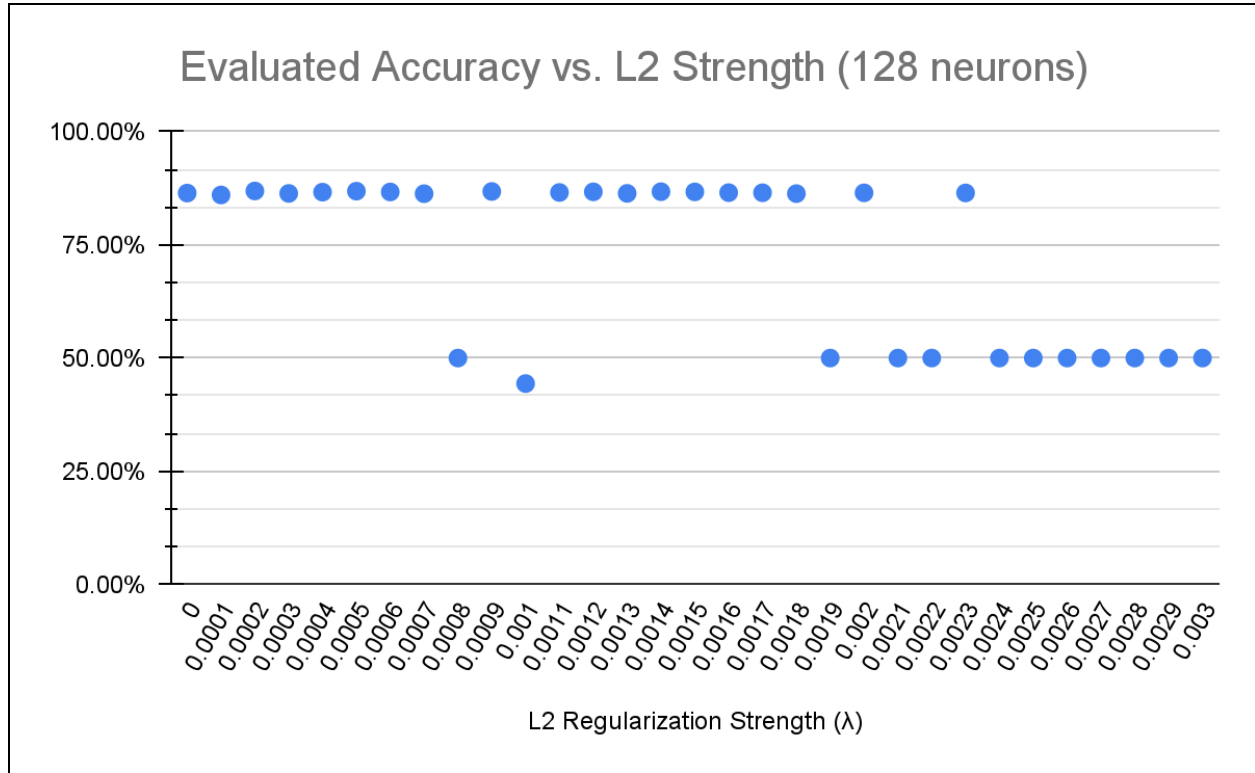


Figure 4.1.4 Scatterplot of evaluated accuracy for network of width 128

Beyond the spread, the median values of both **table 4.1.1** and **4.1.2** suggest that increasing the layer width past 256 neurons does not improve performance, but potentially degrades it. Thus, while wider networks have the capacity for better performance, their optimization stability decreases sharply along with their reliability.

Furthermore, the standard deviation of all networks' accuracies are calculated to measure the true influence of all data points on network stability. Because the interquartile range is a measure of central consistency and uses the median in its calculations, it does not account for the sporadic drops in accuracy. And hence, cannot provide a precise view of the situation. Like calculating the quartiles in the tables before, only values before maximum λ (inside of the "safe zone") are used in calculation. Furthermore, the standard deviation of loss is not calculated due to the presence of numerous extreme outliers on top of sporadic ones that will skew the results positively.

Neurons per Layer	Standard Deviation of Evaluated Accuracy
64	0.0783
128	0.1567
256	0.1128
512	0.1491

1024	0.1678
------	--------

Table 4.1.3 Standard deviation of evaluated accuracy

The standard deviation is the lowest for a network of width 64, making it the most stable in terms of evaluated accuracy, while the network of width 1024 is the least stable because of its much higher standard deviation. However, unlike the interquartile range, the standard deviation does not increase steadily along with network width. That is, the network with width of 128 has a greater standard deviation than both networks of width 256 and 512. This is possibly due to the many outliers it has, as shown in **table 4.1.2**.

Additionally, the network with width of 1024's interquartile range for evaluated accuracies (0.1993) is almost eighty three times that of width 64's interquartile range (0.0043), but its standard deviation of 0.1678 is only around double that of width 64's. These two ratios' imbalance shows that the true influence of all data points makes the network with a width of 1024 far more stable than what the interquartile range alone suggests. However, the data in **table 4.1.3** still complies to the previous observation of reliability and stability decreasing as network width increases.

4.2 - Measurements of Central Tendency for Magnitude of Overfitting

To determine the magnitude of overfitting for the different networks, the difference between the **evaluation loss** and **training loss** is calculated. This difference is called the "generalisation gap" and is a formal indicator for overfitting in machine learning (Nagler, 2024). The following tables show the generalisation gap for networks without and with regularisation, respectively.

Neurons per Layer	Mean Training Loss	Mean Evaluation Loss	Generalisation Gap (difference of means)
64	0.2921	0.3239	0.0318
128	0.2994	0.3467	0.0473
256	0.3064	0.3644	0.0580
512	0.3170	0.3807	0.0637
1024	0.3351	0.4025	0.0674

Table 4.2.1 Comparison of generalisation gap to width of networks *without regularisation*

Neurons per Layer (width of network)	Median Training Loss	Median Evaluation Loss	Generalisation Gap (difference of medians)
64	0.2693	0.3609	0.0916
128	0.2983	0.3530	0.0616
256	0.3422	0.3593	0.0096
512	0.4635	0.4064	-0.0571
1024	0.4561	0.3793	-0.0881

Table 4.2.2 Comparison of generalization gap to width of networks *with regularisation*

To minimise random and sampling errors in data collection, ten more tests are conducted for each network with λ set to zero. The mean values are calculated and entered for **table 4.2.1** as there is an absence of underfitting and extreme outliers. Additionally, more tests minimise the chance for confirmation bias, where one set of values is chosen because it conforms to the predicted trend.

In **table 4.2.2**, the median values from the first test are used due to the presence of outliers before the cutoff. These outliers occur mostly due to one or two instances of 50% accuracy, which are not consistent enough to be an indication for underfitting or counted for cutoff. However, like when calculating for values in **section 4.1**, values past the maximum λ are omitted to focus on the functional zone during machine learning.

As shown in **table 4.2.2**, L2 regularisation is the most effective at reducing the generalisation gap (severity of overfitting) for wider networks with more neurons. Below, **figures 4.2.2** and **4.2.3** show the generalisation gap versus neurons per layer for present and absent regularisation.

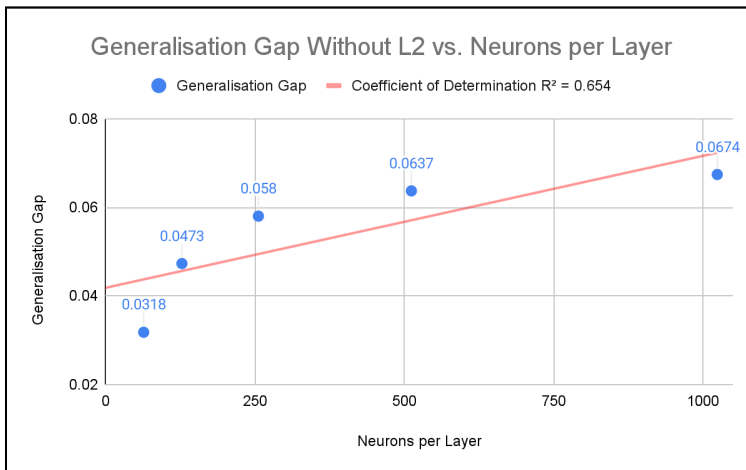


Figure 4.2.2 Generalisation gap without L2

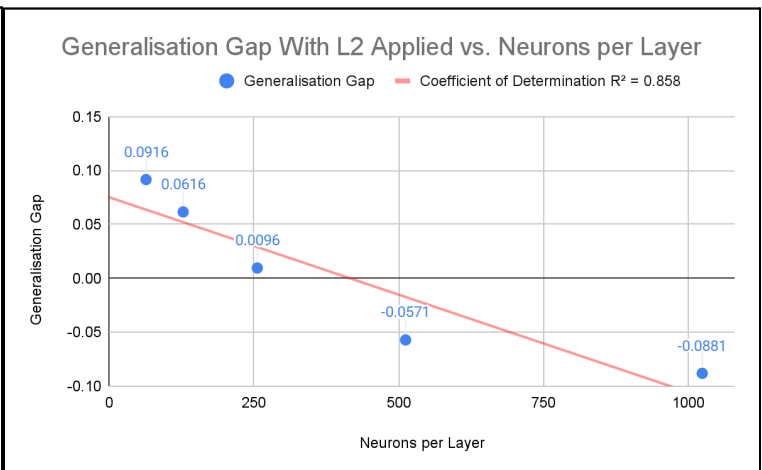


Figure 4.2.3 Generalisation gap with L2

Neurons per Layer (width of network)	Δ Magnitude of Overfitting (difference of generalisation gaps)
64	0.0598
128	0.0143
256	-0.0484
512	-0.1208
1024	-0.1555

Table 4.2.3 Difference between generalisation gaps

To find the effectiveness of L2 regularisation on a model's overfit, the difference between generalisation gaps are calculated as shown in **table 4.2.3** above. This directly measures how much impact L2 has on the model, and utilises the following equation:

$$\Delta \text{Magnitude of Overfitting} = \text{Generalisation Gap}_{\text{with L2}} - \text{Generalisation Gap}_{\text{without L2}}$$

Since the width of the generalization gap is a direct measure of the magnitude of overfitting, a change in the generalization gap reflects a corresponding change in the magnitude of overfitting.

The following graph shows a strong, negative relationship because of its correlation coefficient of -0.914. Additionally, its coefficient of determination R^2 is 0.836, which means that approximately 84% of the change in magnitude of overfitting due to L2 regularisation can be attributed to an increase in network width. Furthermore, the correlation coefficient R is negative 0.917 – indicating a strong, negative correlation between the two variables.

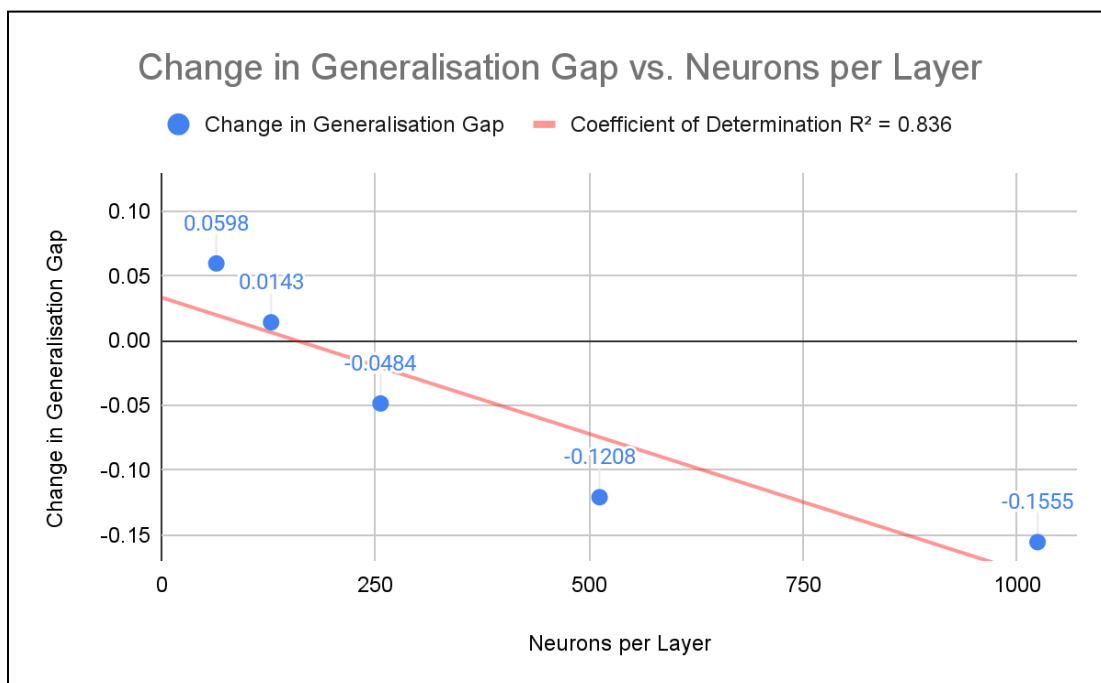


Figure 4.2.4 Change in generalisation gap (magnitude of overfitting)

4.3 - Diagnosing Non-linearity With Residuals

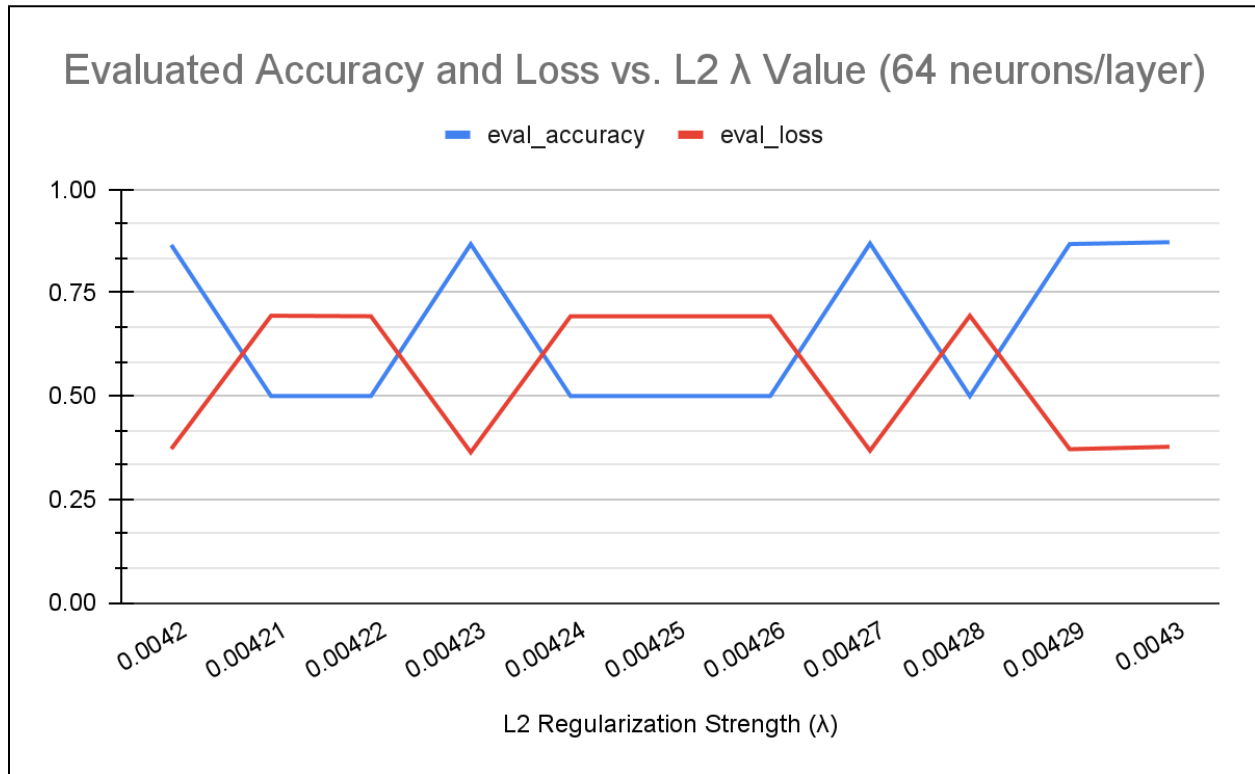


Figure 4.3.1 Specific test of metrics versus regularisation strength in network of width 64

Since the data from the general test (shown in **figure 4.1.1**) is continuous, the maximum L2 value – the value before underfitting – could be any number ranging from it to its succeeding value (e.g. value is displayed as 0.0042 but is actually 0.00423). Thus, a second/specific test is needed to ensure better precision of data. This still follows the rule of three consecutive instances of underfitting as explained in **section 3.0**. And as shown in **figure 4.1.2**, the precise maximum λ for a network with width of 64 neurons is 0.00423. Although the values of the specific test are still continuous, they are more precise by a factor of 10.

By plotting the maximum λ values for each network, a negative correlation is shown. Using linear regression, its trendline's equation is found to be $y = -0.00000343 \cdot x + 0.00293$. The residuals plot in **figure 4.3.2** below shows the difference between the actual values versus the values the linear equation predicted.

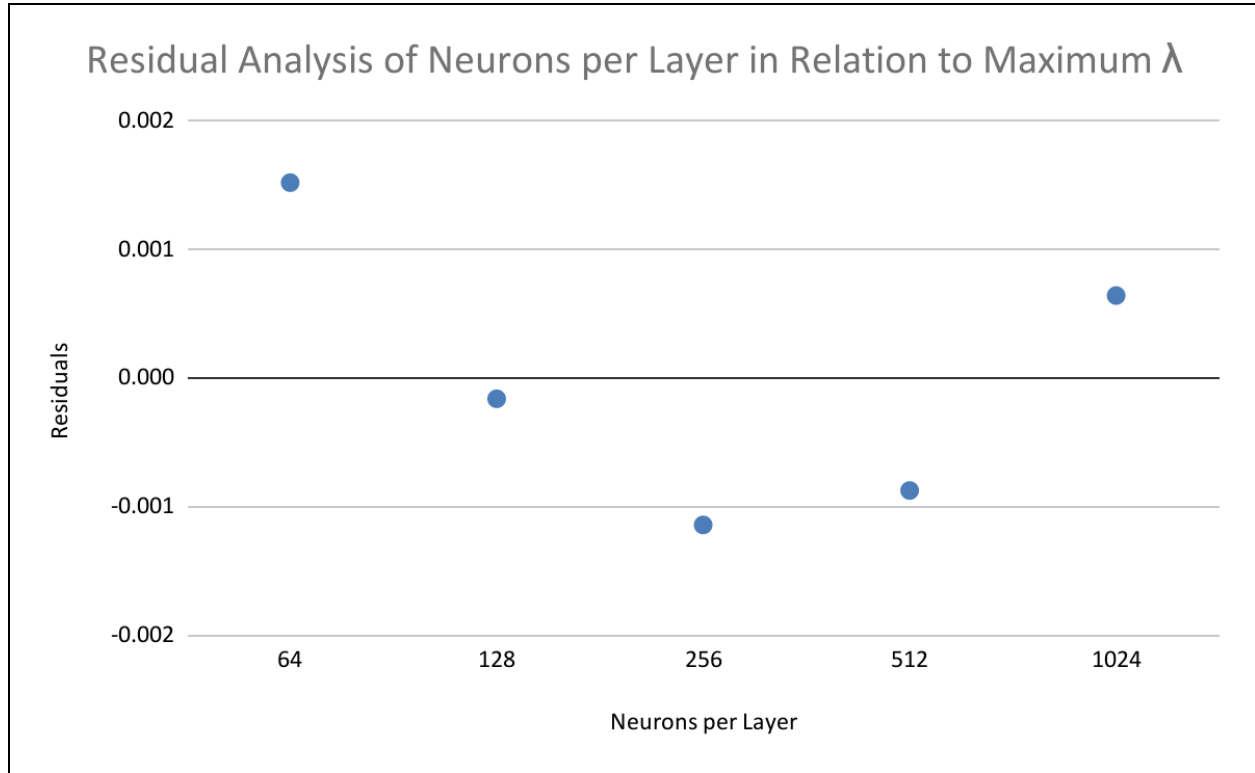


Figure 4.3.2 Residuals of predicted versus actual maximum λ values

However, constant variance is not met because the residuals are not randomly or evenly scattered around $y = 0$. In addition, a wave-like pattern is visible in the residuals, which strongly suggests non-linearity in the relationship between maximum λ values and model width; along with systematic bias. Because of this, linear regression cannot accurately determine the line of best fit for this relationship. Consequently, an exponential decay model is implemented and provides a better functional form for the data. The resulting equation is found using non-linear least squares regression: $y = 0.0071e^{-0.0083x}$, with an R^2 value of 0.964. The following figure shows the datapoints with the exponential equation.

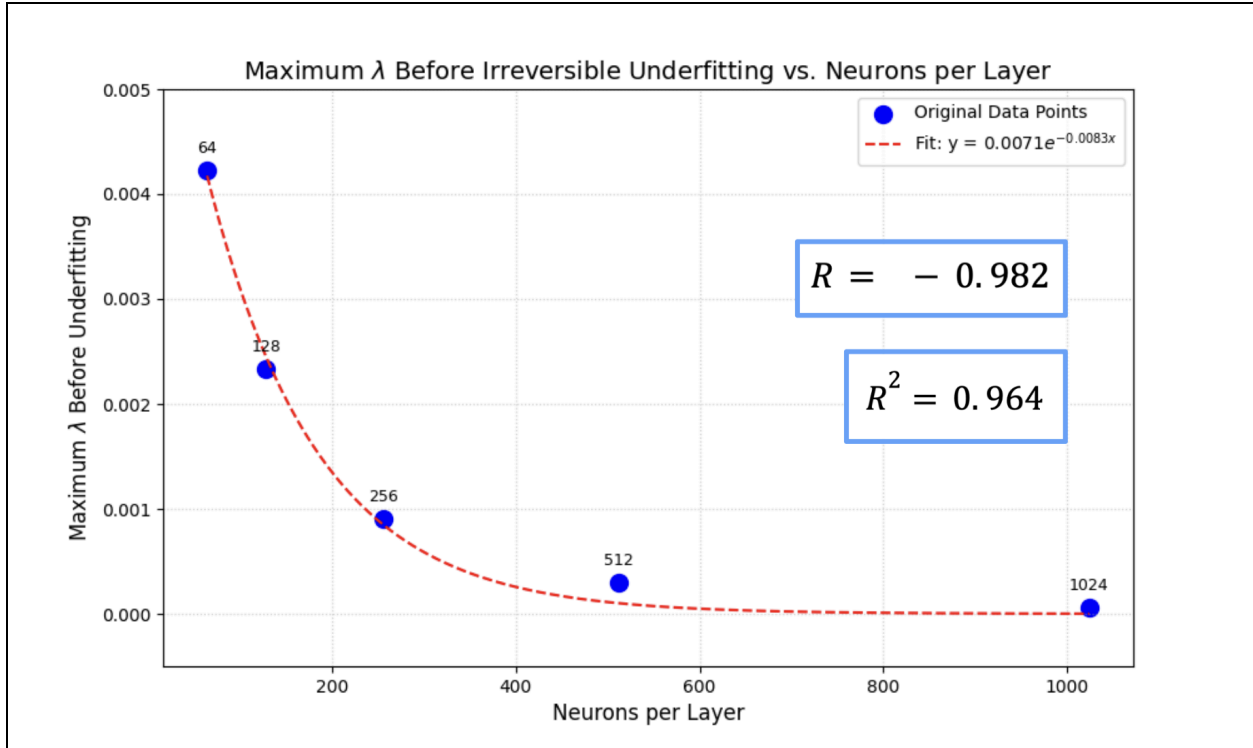


Figure 4.3.3 Maximum λ before irreversible underfitting with exponential decay model made with *Matplotlib for the Python Language* (Hunter, 2007)

Section 5: Conclusion

By finding the relationship between the maximum L2 strength and network width, a lot of unnecessary trial-and-error can be eliminated when trying to find an optimal value via arbitrary guesses during deep/machine learning. Not only will these extra trials waste time, but also contribute to increased energy consumption and greenhouse gas emissions. The degree of impact depends on how large of a scale the training is (Siddik et al, 2025).

The data in **section 4.3** shows a clear, exponentially negative correlation between network width and maximum λ value. This allows for a much better estimation of the valid λ ranges to test during training, via interpolation and extrapolation using the equation:

$y = 0.0071e^{-0.0083x}$. However, the limitation is that the network's depth must remain 4 layers for successful estimation, as the experiment listed depth as a control variable. Likewise, different datasets may yield different results due to the IMDB dataset being the only one used during training.

With a correlation coefficient of -0.914 – shown in **figure 4.2.4** – there is a strong correlation between increase in network width and the positive impact of L2 regularization at reducing the magnitude of overfitting. However, it cannot be concluded that L2 regularisation is effective for all networks, since **figure 4.2.4** shows the smaller networks having an increase in

generalisation gap instead of decrease. Additionally, there is a risk of confirmation bias, since there exists knowledge beforehand that L2 regularisation is the most effective when used on larger networks – as they tend to overfit more (Fang et al, 2020). Therefore, the strong negative correlation applies specifically to wider networks; it does not support a general conclusion regarding the effectiveness of L2 regularisation for all network widths, and the interpretation must remain cautious to mitigate potential confirmation bias.

Furthermore, another potential bias is selection bias. By excluding the data points that lead to underfitting during calculations in **sections 4.1** and **4.2**, it is being artificially restricted to the "good" or "functional" region. While this helps analyze the stable performance zone, it can positively skew the results (like the median or standard deviation) and underestimate the true instability or unreliability of wider networks, especially when exposed to strong regularization.

Future steps in this field of research can include exploring different datasets, model depth – how many layers of neurons in the network – versus L2 regularisation, other forms of regularisation in deep learning (dropout, L1, implicit methods, etc) versus network architecture, and energy usage during training versus network size.

Section 6: References and Bibliography

- Kazdin, A. E., & Tuma, A. H. (1982). Single-case research designs.
<https://sites.hofstra.edu/jeffrey-froh/wp-content/uploads/sites/86/2019/11/Single-Case.pdf>
- Lobo, M. A., Moeyaert, M., Baraldi Cunha, A., & Babik, I. (2017, July). *Single-case design, analysis, and Quality Assessment for Intervention Research*. Journal of neurologic physical therapy : JNPT.
<https://pmc.ncbi.nlm.nih.gov/articles/PMC5492992/>
- Lee, T. B. (2024, November 5). Why the deep learning boom caught almost everyone by surprise. <https://www.understandingai.org/p/why-the-deep-learning-boom-caught>
- Maity, A. (2025, November 9). *Why many ML hyperparameters are powers of 2?* | by Atanu Maity | Medium. Why Many ML Hyperparameters Are Powers of 2?
<https://medium.com/@amrkmc/why-many-ml-hyperparameters-are-powers-of-2-50bbe937c2ba>
- McCaffrey, J. (2023, June 15). *Binary classification using a scikit Neural Network -- Visual Studio Magazine*. Binary Classification Using a scikit Neural Network.
<https://visualstudiomagazine.com/articles/2023/06/15/scikit-neural-network.aspx>

- Yıldırım, S. (2020, March 25). *Problem: Overfitting, solution: Regularization*. Towards Data Science.
<https://towardsdatascience.com/problem-overfitting-solution-regularization-b466d3d1f4da/>
- Zhu, A. (2022, June 20). *What is Regularization: Bias-Variance Tradeoff*. Towards Data Science.
<https://towardsdatascience.com/machine-learning-bias-variance-tradeoff-and-regularization-94846f945131/>
- Alexandrov, A. (n.d.). *A group of oranges with water drops on them*. 123RF.
https://www.123rf.com/photo_207344518_a-group-of-oranges-with-water-drops-on-them.html
- Vera, J. (2022, June 11). *L1 & L2 regularization, plus dropout! | by Juan Vera | Medium*. Medium.
<https://medium.com/@vxnuaj/l1-l2-regularization-plus-dropout-a1e84e65c464>
- Inamdar, A. (2025, July 7). *Understanding Model Depth in Machine Learning: Two Perspectives*. Medium.
<https://medium.com/@inamdaraditya98/understanding-model-depth-in-machine-learning-two-perspectives-b27ae2c0d3ac>
- Pehlevan, C., & Bordelon, B. (2023, June). *Lecture Notes on Infinite-Width Limits of Neural Networks*.
https://pehlevan.seas.harvard.edu/sites/g/files/omnuum6471/files/pehlevan/files/princeton_lecture_notes.pdf
- Topper, N. (2025, July 10). *Sigmoid activation function: An introduction*. BuiltIn.
<https://builtin.com/machine-learning/sigmoid-activation-function>
- Srivastava, T. (2025, May 1). *12 Important Model Evaluation Metrics for Machine Learning Everyone Should Know (Updated 2025)*. Analytics Vidhya.
<https://www.analyticsvidhya.com/blog/2019/08/11-important-model-evaluation-error-metrics/>
- Alake, R. (2024, December 4). *Loss Functions in Machine Learning Explained*. DataCamp.
<https://www.datacamp.com/tutorial/loss-function-in-machine-learning>
- Kumar, R. (2025, May 29). *What is Underfitting? How to Detect and Overcome High Bias in ML Models*. DataCamp. <https://www.datacamp.com/tutorial/what-is-underfitting>

- Kromydas, B. (2023, January 3). *Training Neural Networks for beginners*. LearnOpenCV. <https://learnopencv.com/how-to-train-neural-networks-for-beginners/>
- Jang, J. (2022, February 3). *ML-How to choose Lambda*. Medium. https://medium.com/@shiny_jay/ml-how-to-choose-lambda-9d0000c1491c
- Zhao, D. (2023, June 1). *Combining Explicit and Implicit Regularization for Efficient Learning in Deep Networks*. arXiv.org. <https://arxiv.org/pdf/2306.00342>
- Kumar, A. (2022, May 17). *Regularization in machine learning: Concepts & examples*. Analytics Yogi. <https://vitalflux.com/regularization-in-machine-learning-concepts-examples/>
- Nagler, T. (2024, July 7). *Statistical Learning Theory*. Ludwig Maximilian University of Munich. <https://tnagler.github.io/slt-2024.pdf>
- Python Numerical Methods*. Least Square Regression for Nonlinear Functions - Python Numerical Methods. (n.d.). <https://pythonnumericalmethods.studentorg.berkeley.edu/notebooks/chapter16.05-Least-Square-Regression-for-Nonlinear-Functions.html>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- Abadi, Martín et al. (2015). *TensorFlow: Large-scale machine learning on heterogeneous systems*. TensorFlow. <https://tensorflow.org>
- Siddik, M., Shehabi, A., & Marston, L. (2021, May 21). *The environmental footprint of data centers in the United States*. IOP Science. <https://iopscience.iop.org/article/10.1088/1748-9326/abfb1>
- Yu, Norran. (2025, November). *MDM4U_statistics_project [Code]*. GitHub. <https://github.com/Cel3brimbor/MDM4U>

Section 7: Appendices

7.1 - Appendix A

The materials for the MDM4U statistics project are provided below for replication purposes. The experiment repository (code used in testing neural networks and gathering data) and data files are hosted externally.

Experiment Repository:

The full code used for data creation, collection, and visualization can be accessed at the following public GitHub repository:

https://github.com/Cel3brimbor/MDM4U_statistics_project

7.2 - Appendix B

Project Data and Graphs:

The compiled project data and all generated figures are available in the linked resource:

 [Statistics Project Data & Graphs](#)

 [Residual Plot for Statistics Project](#)

All data in the sheets are gathered from neural network testing results. See **section 7.1 - Appendix A** for the external repository.